

Exigences de test

INDICE A / 26 MAI 2025

Ce document définit la stratégie de validation de la qualité fonctionnelle, technique et UX de l'application, en couvrant les différents types de tests à mener, les cas clés, les critères de validation, ainsi que les objectifs de performance à atteindre avant toute mise en production.

Stratégie de test

Definition of Done

https://docs.google.com/document/d/11N37QoPueA9gL0hkv89S_-oSqcUjCy_pgYsFkVcHUAA/edit?mode=html&tab=t.0

Tests unitaires cf: Definition of done

Tests d'intégration cf: Definition of done

Tests fonctionnels

Les **tests fonctionnels** ont pour but de valider que les fonctionnalités de l'application répondent **aux besoins métiers réels** et fonctionnent comme attendu **du point de vue de l'utilisateur final**. Ces tests s'appuient sur des **scénarios d'usage complets**, exécutés à travers l'**interface utilisateur**.

Objectif

- Vérifier que chaque parcours utilisateur (inscription, apprentissage, simulation, progression, récompense, etc.) **reproduit fidèlement le comportement attendu**, tel que défini dans les spécifications fonctionnelles.
- Détecter les **bugs fonctionnels**, incohérences d'interface ou erreurs de navigation qui ne sont pas visibles via les tests unitaires ou d'intégration.

- **Simuler des sessions réelles** de jeu et d'apprentissage pour évaluer l'expérience utilisateur dans son ensemble.

Les tests fonctionnels se concentrent sur :

- **L'interaction avec les composants visibles** (boutons, champs, animations, notifications)
- **La logique métier déclenchée par l'utilisateur** (calculs, vérifications, validations)
- **Les transitions d'écran ou d'états** (progression de niveau, accès aux fonctionnalités déverrouillées)
- **Les cas d'erreur utilisateur** (réponses incorrectes, champs invalides, actions interdites)

Exemples de scénarios fonctionnels pour le projet

Scénario 1 : L'utilisateur simule un investissement en actions

1. L'utilisateur ouvre l'écran "Simulation".
2. Il sélectionne "Actions" comme type d'investissement.
3. Il saisit un montant de 1 000 €.
4. Il valide la simulation.
5. Le système affiche le rendement et met à jour le portefeuille.
6. L'interface affiche un message de succès.

Vérifications attendues :

- Le champ de montant accepte uniquement des nombres positifs.
- Le bouton "Simuler" est désactivé tant que les champs ne sont pas remplis.
- Le calcul du rendement est exact.
- La récompense de progression est déclenchée si applicable.

Scénario 2 : L'utilisateur termine un quiz sur le budget

1. L'utilisateur accède au module éducatif "Budget".
2. Il clique sur "Commencer le quiz".
3. Il répond à 5 questions.
4. Il valide ses réponses.
5. Le système affiche le score, la correction, et met à jour son profil.

Vérifications attendues :

- Les bonnes questions sont affichées pour le module concerné.
- La correction est affichée immédiatement après la validation.
- Le score est enregistré dans la base utilisateur.
- Si l'utilisateur obtient ≥ 80 %, il passe au module suivant.

Tests de régression

Les **tests de régression** sont essentiels pour garantir que l'ajout de nouvelles fonctionnalités ou les modifications du code **n'affectent pas négativement les fonctionnalités existantes**. Ils permettent de détecter rapidement si une mise à jour casse un comportement précédemment validé, en particulier dans un projet modulaire et évolutif comme le tien.

Objectif

- **Prévenir les régressions fonctionnelles** après chaque évolution du produit.
- **Valider l'intégrité globale de l'application** suite à des changements (nouvelle fonctionnalité, refactoring, correction de bug).
- Réduire le risque d'introduire des **effets de bord invisibles** qui échappent aux tests manuels ou unitaires.

Fonctionnement

- Les tests de régression sont constitués d'un **ensemble de scénarios critiques déjà validés** (tests unitaires, intégration, fonctionnels).
- Ces tests sont **automatisés** et **exécutés systématiquement** à chaque :
 - *Merge* dans la branche principale
 - *Pull request* (PR)
 - *Déploiement de préproduction ou production*
- Leur exécution est intégrée dans la **pipeline CI/CD** (par ex. GitHub Actions, GitLab CI, Jenkins).

Exemples de régressions typiques à éviter dans le projet

Cas 1 – Récompense bloquée

- Lorsqu'on ajoute un nouveau type d'investissement (ex. matières premières), la condition de déverrouillage des fonctionnalités avancées (ex. cryptoactifs) ne fonctionne plus pour les actions classiques.

Cas 2 – Quiz non enregistré

- Après la mise à jour d'un module éducatif, les réponses des quiz ne sont plus persistées dans le profil utilisateur.

Cas 3 – Portefeuille corrompu

- Suite à une modification dans la logique d'ajout d'actif, le solde du portefeuille devient négatif ou incohérent.

Implémentation technique

- Outils recommandés :
 - **Jest** pour relancer tous les tests unitaires + d'intégration
 - **Cypress / Detox** pour les tests UI critiques
 - **Snapshots** ou tests d'état pour détecter des changements visuels ou inattendus

Vue d'ensemble des cas de test

Fonctionnalité

Cas de test principaux

Simulation d'investissement	Vérifier les calculs de rendement, les limites d'entrée, les erreurs attendues
Quiz éducatif	Tester la génération aléatoire des questions, la validation des réponses, les scores
Déverrouillage de fonctionnalités	Vérifier que les conditions métier débloquent les bons modules
Portefeuille virtuel	S'assurer que l'ajout/suppression d'actifs est cohérent et reflété dans l'interface
Événements aléatoires	Tester les effets sur les actifs et la réaction des modules concernés
Récompenses et progression	Valider le système de points, l'historique utilisateur, et les niveaux atteints

| Critères d'acceptation

Chaque fonctionnalité est considérée comme **acceptable** si les conditions suivantes sont remplies :

- 100 % des tests unitaires et d'intégration **passent sans erreur**
- Fonctionnalité testée dans **tous les scénarios définis**, y compris cas limites et erreurs
- Aucun **bug bloquant ou critique** détecté lors des tests fonctionnels
- Comportement **conforme aux spécifications fonctionnelles**
- Performance respectant les référentiels définis ci-dessous

| Indicateurs de qualité

Indicateur	Objectif
Taux de couverture des tests	≥ 90 % pour les modules critiques, ≥ 90 % globalement
Taux de bugs bloquants	0 % avant validation finale
Temps moyen de réponse API	< 500 ms pour les appels critiques
Taux de réussite des scénarios UI	100 % sur les parcours utilisateur majeurs
Nombre de régressions détectées	≤ 1 par version majeure

| Référentiels de performance

Test	Objectif
Temps de chargement initial	< 2 secondes sur un appareil milieu de gamme
Temps de réponse API	< 500 ms pour les endpoints portefeuille, quiz, recompenses
Capacité simultanée	500 utilisateurs connectés sans dégradation visible de performance

Framerate moyen (mobile) ≥ 30 FPS pour les écrans animés et les graphiques interactifs

Utilisation mémoire
(mobile) < 200 MB en usage standard

